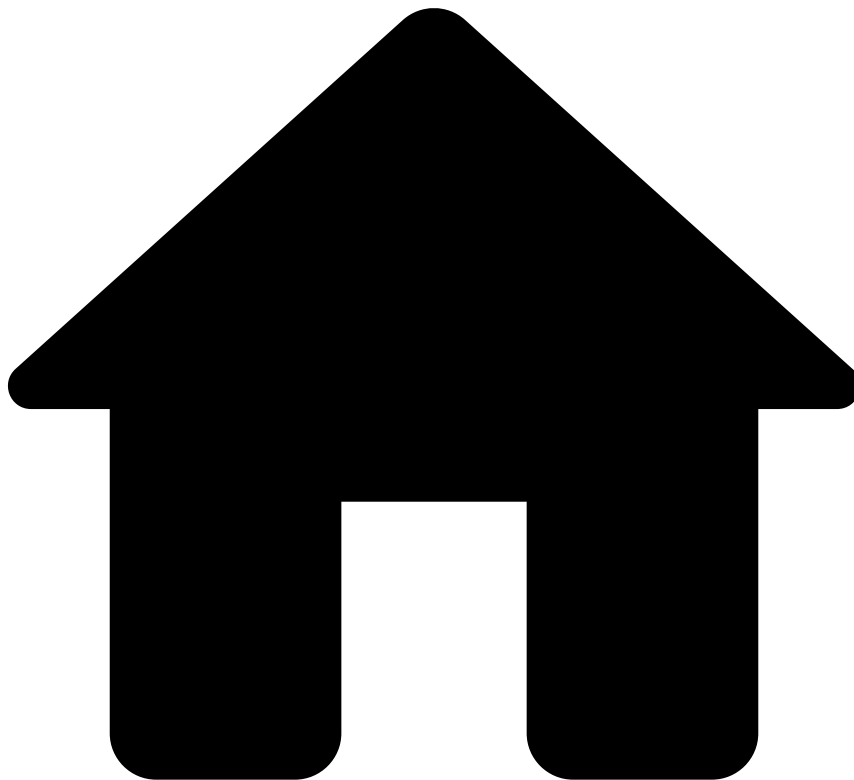


•



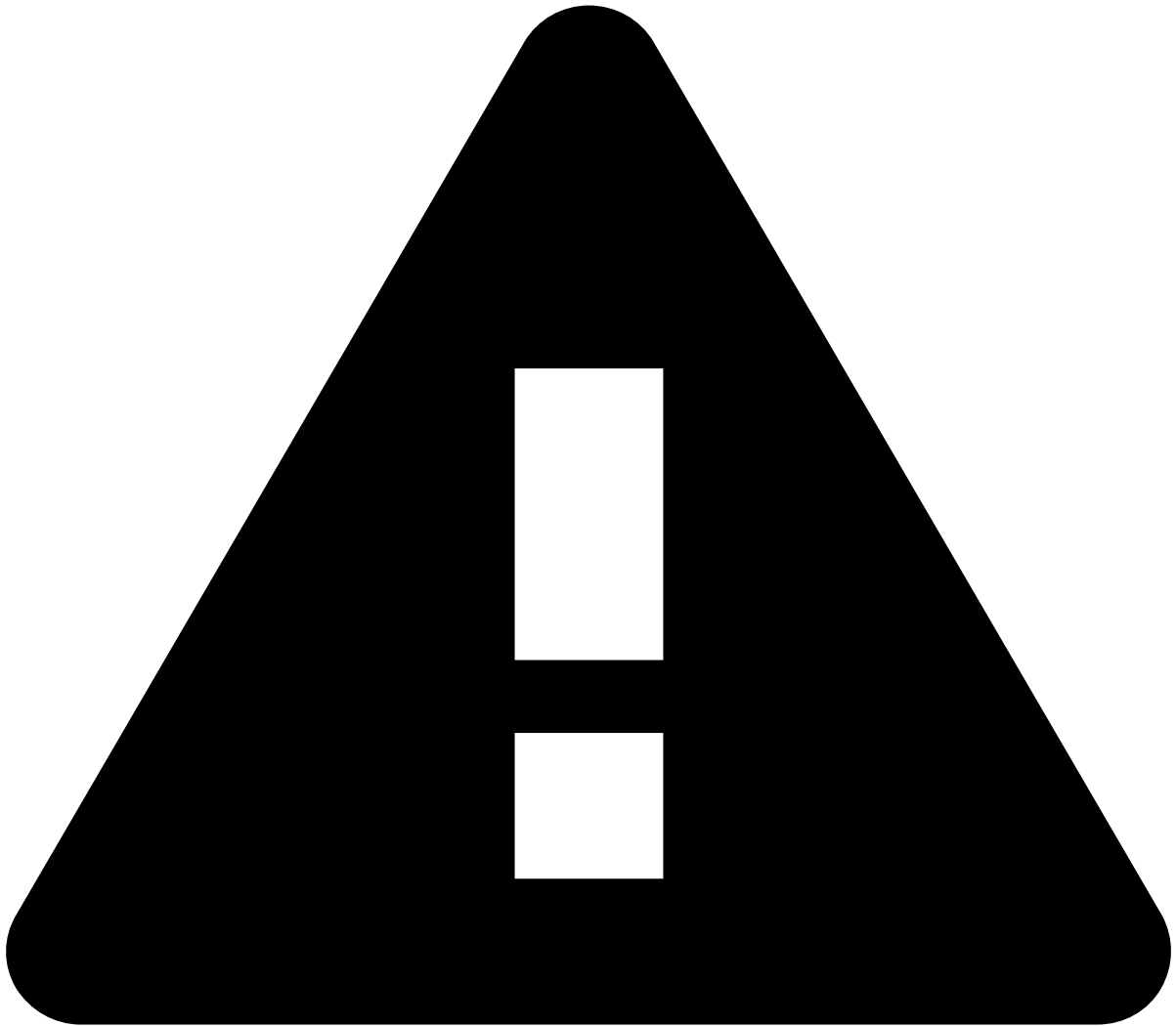
- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- LightGBM Locale Issues

On this page

LightGBM Locale Issues

Was this page helpful? 

When using versions older than LightGBM v3.2.0 or Pulse 22.0, there may be wrong scores during prediction due to locale mismatches between train and prediction environments.



caution

This caveat applies to versions **older** than Pulse 22.0, that includes a patched version of LightGBM, so that there are no locale issues

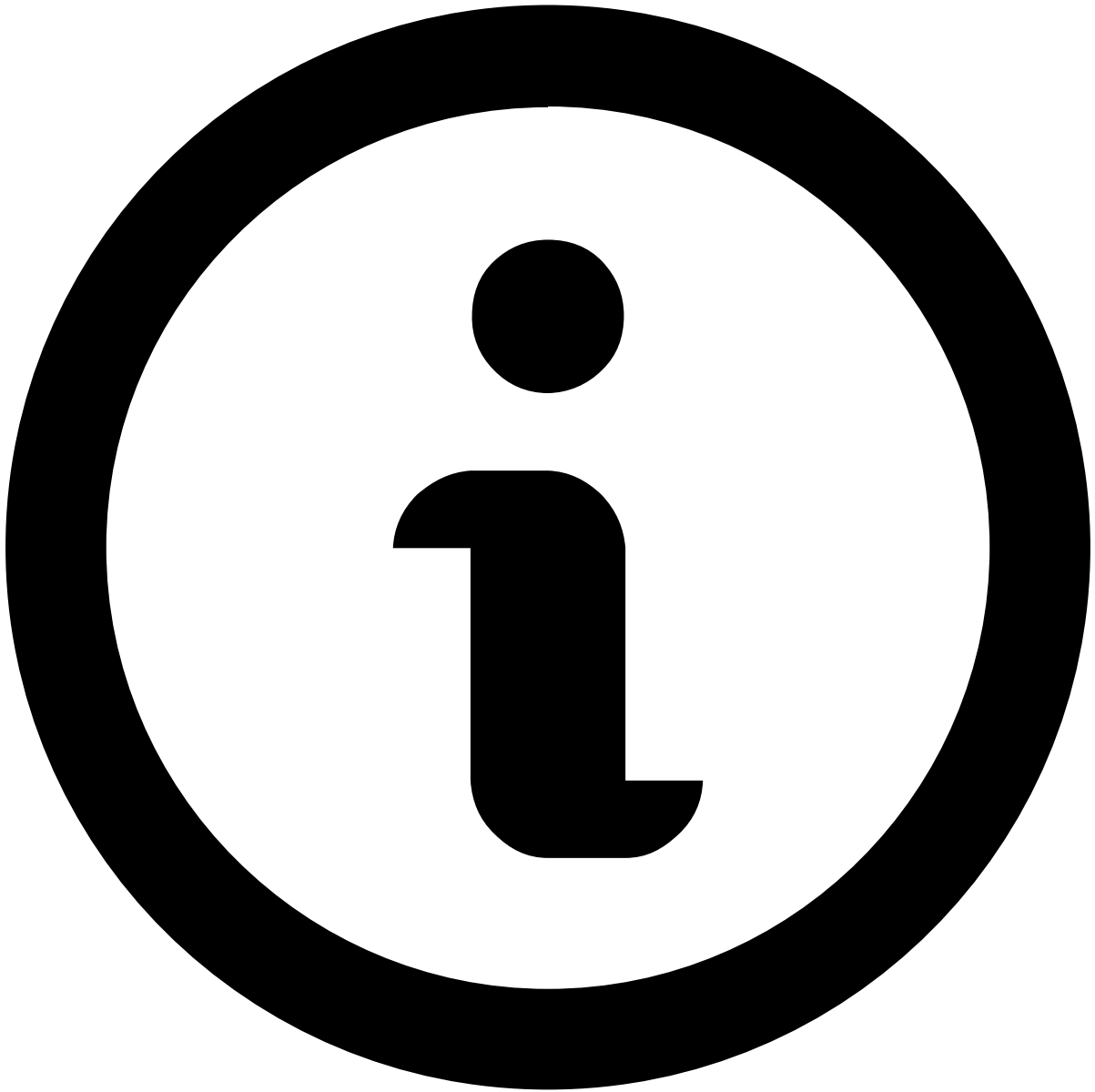
The fix is also present in all of the following Pulse hotfix versions:

- `>= 19.1.41`
- `>= 20.1.18`
- `>= 20.3.20`
- `>= 21.1.3`

When importing models that were trained with the help of Java in some way, follow this rule:

- When training LightGBM models outside Pulse using Java tools, enforce locale `LC_ALL=C` during model training to avoid broken scores.

This is true whether using a Java LightGBM provider such as MMLSpark, or any LightGBM provider such as Python's, together with Java tools like JEP.



note

This rule can only be ignored if you know your LightGBM provider uses the core LightGBM library release v3.2.0 or later.

Explanation¹

If Java is not using the default **C** locale, it will break LightGBM's, producing a LightGBM model file that will be read back incorrectly.

Although LightGBM in Pulse uses Java as well, it no longer suffers from this issue since we patched LightGBM and are using that patch, that will come out in the LightGBM v3.2.0 release.

How to control the locale?²

Depending on where the job is running, there are several ways to override a locale. If running a local process prepend `LC_ALL=C` to your command or do `export LC_ALL=C` before using your command. If you are using Spark, pass the Spark property `spark.executorEnv.LC_ALL=C`.